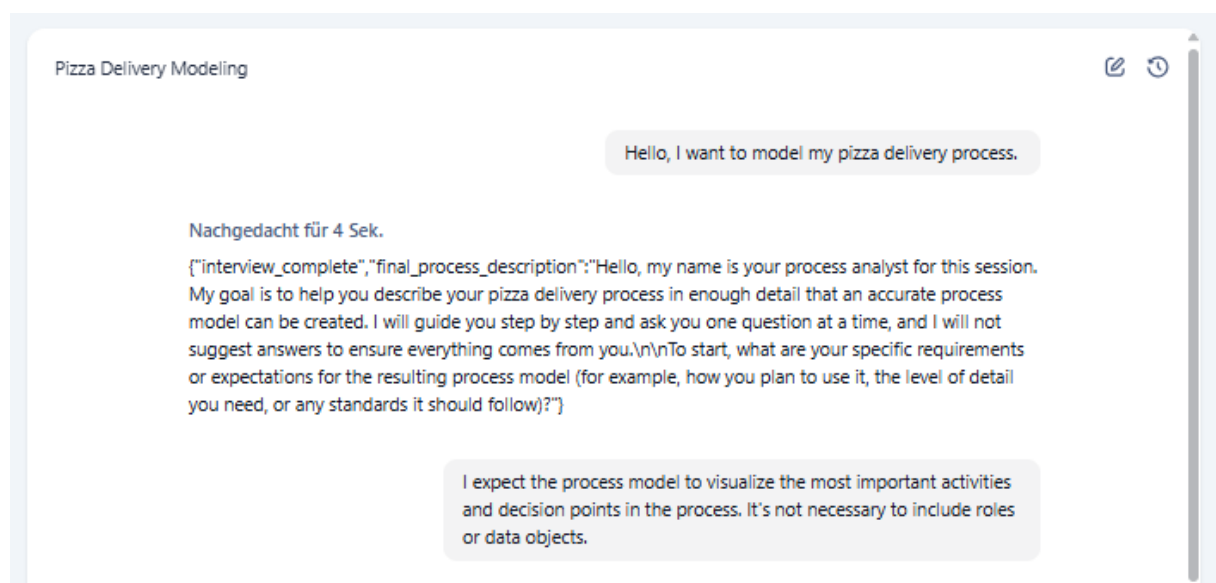
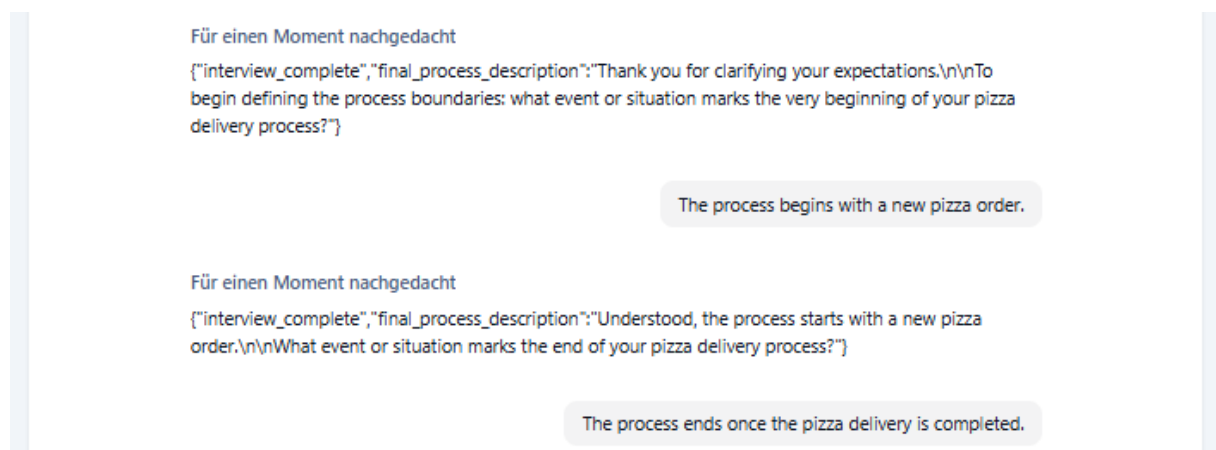


Example of Dialog-to-Process Model Transformation

When a user initiates a conversation in the chat interface, the Interview Agent introduces itself and briefly explains the interview workflow, including how the conversation will proceed and when the user will be asked to confirm intermediate results. Immediately after this onboarding, the agent requests the user's requirements for the resulting process model, for example what level of detail is desired and whether the focus lies on the main flow or also on exceptions and optional information. This initial step is intentionally placed at the beginning to align the subsequent discovery with the intended modeling purpose and to avoid collecting irrelevant details.



Once the requirements are clarified, the Interview Agent conducts the interview in a sequence consistent with the structure proposed by Dumas et al. 2018. The agent asks one question at a time and waits for the user's response before continuing, which improves readability and reduces cognitive load for the interviewee. The discovery starts with process boundaries, ensuring that start and end points as well as the scope of the process are clearly defined. It then moves to the main activities and their ordering to establish the core control flow.



Für einen Moment nachgedacht >

```
{"interview_complete":true,"final_process_description":"Thank you, that defines the start and end of the process.\n\nAfter a new pizza order is received, what is the very first activity that is performed in your process?"}
```

After the order is received, the first task is to make the pizza.

Für einen Moment nachgedacht

```
{"interview_complete":true,"final_process_description":"Thank you. After the pizza is made, what is the next activity performed in the process?"}
```

Then, the delivery route is assessed.

Nachgedacht für 3 Sek. >

```
{"interview_complete":true,"final_process_description":"After the delivery route is assessed, what is the next activity that happens in your pizza delivery process?"}
```

If the delivery route is less than one kilometer, the next task is to deliver the pizza by foot.

Für einen Moment nachgedacht

```
{"interview_complete":true,"final_process_description":"Thank you. You mentioned delivering by foot if the route is less than one kilometer.\n\nWhat happens if the delivery route is one kilometer or more—what is the next activity in that case?"}
```

In this case the pizza is delivered by bike.

Für einen Moment nachgedacht >

```
{"interview_complete":true,"final_process_description":"Understood: if the route is less than one kilometer, the pizza is delivered by foot; otherwise it is delivered by bike.\n\nAfter the pizza has been delivered (whether by foot or by bike), is there any further activity in the process before you consider the delivery completed, or does the process end immediately at that point?"}
```

After that the delivery is considered complete and the process ends.

Depending on the user's answers and the previously stated modeling requirements, the agent optionally collects additional details such as involved roles, handoffs, and supporting systems or documents, but only when such information is relevant for the target process model. Throughout the interaction, the agent asks clarifying follow-up questions whenever an answer is ambiguous, incomplete, or implies alternative flows that have not yet been specified. Importantly, this clarification behavior remains user-response-driven, the LLM does not infer missing process elements silently but instead elicits them explicitly, thereby improving transparency and completeness.

Nachgedacht für 4 Sek. >

```
{"interview_complete":true,"final_process_description":"Thank you, that clarifies the current end of the process.\n\nBetween receiving the new pizza order and considering the delivery complete, are there any other important decision points or alternative paths that occur in your process, or is the route-based choice between delivery by foot and by bike the only decision point you want to include?"}
```

Yes, that's the only decision point in that process.

Nachgedacht für 3 Sek. >

```
{"interview_complete":true,"final_process_description":"Understood.\n\nApart from the steps you already mentioned, are there any additional activities you want to include between receiving the new pizza order and making the pizza, or between making the pizza and assessing the delivery route?"}
```

No, there are no additional steps.

Für einen Moment nachgedacht

```
{"interview_complete":true,"final_process_description":"Here is a structured summary of your pizza delivery process as described so far:\n\n1. Start Event \n - The process begins when a new pizza order is received.\n\n2. Make Pizza \n - The first activity after receiving the order is to make the pizza. \n - There are no other activities before or between this step.\n\n3. Assess Delivery Route \n - After the pizza is made, the delivery route is assessed.\n\n4. Route-Based Decision \n - A decision is made based on the distance of the delivery route:\n - If the route is less than one kilometer: the next activity is to deliver the pizza by foot.\n - If the route is one kilometer or more: the next activity is to deliver the pizza by bike.\n\n5. Complete Delivery \n - After the pizza is delivered (by foot or by bike), the delivery is considered complete.\n\n6. End Event \n - The process ends once the pizza delivery is completed.\n\nThere are no additional activities between these steps, and the only decision point in this process model is the route-based choice between delivery by foot and delivery by bike. Roles and data objects are not included in the model, as per your requirements.\n\nIs this description accurate and complete for the pizza delivery process you want to model, or would you like to change or add anything?"}
```

Thank you, the description is accurate.

After the agent determines that the essential process knowledge has been collected, it consolidates the discovered information into a coherent natural language process description. This description functions as an explicit intermediate output that the user can verify before any transformation occurs. The Interview Agent then requests confirmation that the description is accurate and complete, giving the user the opportunity to correct errors, add missing steps, or refine decision logic. Only after the user confirms the description does the system expose the explicit confirmation mechanism, ensuring that the transition to modeling is a deliberate user action rather than an implicit interpretation of agreement.

Für einen Moment nachgedacht >

```
{"interview_complete":true,"final_process_description":"Pizza Delivery Process Description (Final)\n\n1. Start Event\n- The process starts when a new pizza order is received.\n\n2. Make Pizza\n- The first activity is to make the pizza.\n- There are no other activities before or between order receipt and making the pizza.\n\n3. Assess Delivery Route\n- Once the pizza is made, the delivery route is assessed.\n\n4. Route-Based Decision\n- A decision is made based on the distance of the delivery route.\n- If the route is less than one kilometer, the pizza is delivered by foot.\n- If the route is one kilometer or more, the pizza is delivered by bike.\n\n5. Complete Delivery\n- After the pizza is delivered (either by foot or by bike), the delivery is considered complete.\n\n6. End Event\n- The process ends once the pizza delivery is completed.\n\nNotes\n- The only decision point in this process is the route-based choice between delivery by foot and delivery by bike.\n- Roles and data objects are intentionally not included in this model, according to the stated requirements."}
```

Approval required

Please review the final process description and confirm whether it is correct and complete.

If you approve, I will proceed with the next step (creating the process model).
If anything needs to be changed or added, please decline — the interview agent will then ask for the needed adjustments.

Do you approve this final process description? P

Approve

Reject

Once the user clicks “Approve”, the workflow activates the Transformer Agent. The Transformer Agent receives the confirmed process description and generates a corresponding structured JSON representation that encodes the core process elements and their relationships. This intermediate representation is essential for robustness because it separates the natural language process description from formal model generation and enables deterministic validation before further transformations. The generated JSON is then checked by a validation algorithm. If the algorithm detects syntactic issues, the Transformer Agent iteratively revises its output until the JSON is valid. Once validation succeeds, the first JSON representation is stored as the output of the transformation stage, enabling traceability and reproducibility of downstream steps.

```

{
  "process": [
    {
      "type": "startEvent",
      "id": "start",
      "label": "New pizza order received"
    },
    {
      "type": "task",
      "id": "task1",
      "label": "Make pizza"
    },
    {
      "type": "task",
      "id": "task2",
      "label": "Assess delivery route"
    },
    {
      "type": "exclusiveGateway",
      "id": "gateway1",
      "label": "Route less than one kilometer?",
      "has_join": false,
      "branches": [
        {
          "condition": "Route is less than one kilometer",
          "path": [
            {
              "type": "task",
              "id": "task3",
              "label": "Deliver pizza by foot"
            },
            {
              "type": "endEvent",
              "id": "end1",
              "label": "Pizza delivery completed"
            }
          ]
        },
        {
          "condition": "Route is one kilometer or more",
          "path": [
            {
              "type": "task",
              "id": "task4",
              "label": "Deliver pizza by bike"
            },
            {
              "type": "endEvent",
              "id": "end2",
              "label": "Pizza delivery completed"
            }
          ]
        }
      ]
    }
  ]
}

```

The BPMN XML Generator component subsequently reads the validated JSON file and enriches it into a second, extended JSON representation. This enrichment step adds modeling details required for BPMN generation, such as explicit flow specifications that support consistent creation of BPMN elements and sequence flows.

Transformed process:

```
{
  "elements": [
    {
      "id": "start",
      "type": "startEvent",
      "label": "New pizza order received",
      "incoming": [],
      "outgoing": [
        "start-task1"
      ]
    },
    {
      "id": "task1",
      "type": "task",
      "label": "Make pizza",
      "incoming": [
        "start-task1"
      ],
      "outgoing": [
        "task1-task2"
      ]
    },
    {
      "id": "task2",
      "type": "task",
      "label": "Assess delivery route",
      "incoming": [
        "task1-task2"
      ],
      "outgoing": [
        "task2-gateway1"
      ]
    },
    {
      "id": "gateway1",
      "type": "exclusiveGateway",
      "label": "Route less than one kilometer?",
      "incoming": [
        "task2-gateway1"
      ],
      "outgoing": [
        "gateway1-task3",
        "gateway1-task4"
      ]
    },
    {
      "id": "task3",
      "type": "task",
      "label": "Deliver pizza by foot",
      "incoming": [
        "gateway1-task3"
      ],
      "outgoing": [
        "task3-end1"
      ]
    }
  ],
  "end": {
    "id": "end1",
    "type": "endEvent",
    "label": "Pizza delivery completed",
    "incoming": [
      "task3-end1"
    ],
    "outgoing": []
  }
}
```

```

{
  "id": "task4",
  "type": "task",
  "label": "Deliver pizza by bike",
  "incoming": [
    "gateway1-task4"
  ],
  "outgoing": [
    "task4-end2"
  ]
},
{
  "id": "end2",
  "type": "endEvent",
  "label": "Pizza delivery completed",
  "incoming": [
    "task4-end2"
  ],
  "outgoing": []
}
],
"flows": [
  {
    "id": "start-task1",
    "sourceRef": "start",
    "targetRef": "task1",
    "condition": null
  },
  {
    "id": "task1-task2",
    "sourceRef": "task1",
    "targetRef": "task2",
    "condition": null
  },
  {
    "id": "task2-gateway1",
    "sourceRef": "task2",
    "targetRef": "gateway1",
    "condition": null
  },
  {
    "id": "task3-end1",
    "sourceRef": "task3",
    "targetRef": "end1",
    "condition": null
  },
  {
    "id": "gateway1-task3",
    "sourceRef": "gateway1",
    "targetRef": "task3",
    "condition": "Route is less than one kilometer"
  },
  {
    "id": "task4-end2",
    "sourceRef": "task4",
    "targetRef": "end2",
    "condition": null
  },
  {
    "id": "gateway1-task4",
    "sourceRef": "gateway1",
    "targetRef": "task4",
    "condition": "Route is one kilometer or more"
  }
]
}

```

Based on this extended representation, the generator produces a BPMN XML file that encodes the process model in a machine-readable format compatible with standard BPMN tools. To ensure the model can be rendered directly, the XML is then extended with BPMN Diagram Interchange (DI) elements, which define the graphical layout information required for visualization. The resulting final BPMN XML file is stored as the prototype's main output and can be opened and inspected in common modeling environments, for example bpmn.io, to visualize the generated model and assess its structure and readability.

```
<?xml version="1.0" encoding="UTF-8"?>
<bpmn:definitions xmlns:bpmn="http://www.omg.org/spec/BPMN/20100524/MODEL" xmlns:bpmndi="http://www.omg.org/spec/BPMN/20100524/DI"
xmlns:dc="http://www.omg.org/spec/DD/20100524/DC" xmlns:di="http://www.omg.org/spec/DD/20100524/DI" id="definitions_1"
targetNamespace="http://bpmn.io/schema/bpmn">
  <bpmn:process id="Process_1" isExecutable="false">
    <bpmn:startEvent id="start" name="New pizza order received">
      <bpmn:outgoing>start-task1</bpmn:outgoing>
    </bpmn:startEvent>
    <bpmn:task id="task1" name="Make pizza">
      <bpmn:incoming>start-task1</bpmn:incoming>
      <bpmn:outgoing>task1-task2</bpmn:outgoing>
    </bpmn:task>
    <bpmn:task id="task2" name="Assess delivery route">
      <bpmn:incoming>task1-task2</bpmn:incoming>
      <bpmn:outgoing>task2-gateway1</bpmn:outgoing>
    </bpmn:task>
    <bpmn:exclusiveGateway id="gateway1" name="Route less than one kilometer?">
      <bpmn:incoming>task2-gateway1</bpmn:incoming>
      <bpmn:outgoing>gateway1-task3</bpmn:outgoing>
      <bpmn:outgoing>gateway1-task4</bpmn:outgoing>
    </bpmn:exclusiveGateway>
    <bpmn:task id="task3" name="Deliver pizza by foot">
      <bpmn:incoming>gateway1-task3</bpmn:incoming>
      <bpmn:outgoing>task3-end1</bpmn:outgoing>
    </bpmn:task>
    <bpmn:endEvent id="end1" name="Pizza delivery completed">
      <bpmn:incoming>task3-end1</bpmn:incoming>
    </bpmn:endEvent>
    <bpmn:task id="task4" name="Deliver pizza by bike">
      <bpmn:incoming>gateway1-task4</bpmn:incoming>
      <bpmn:outgoing>task4-end2</bpmn:outgoing>
    </bpmn:task>
    <bpmn:endEvent id="end2" name="Pizza delivery completed">
      <bpmn:incoming>task4-end2</bpmn:incoming>
    </bpmn:endEvent>
    <bpmn:sequenceFlow id="start-task1" sourceRef="start" targetRef="task1" />
    <bpmn:sequenceFlow id="task1-task2" sourceRef="task1" targetRef="task2" />
    <bpmn:sequenceFlow id="task2-gateway1" sourceRef="task2" targetRef="gateway1" />
    <bpmn:sequenceFlow id="task3-end1" sourceRef="task3" targetRef="end1" />
    <bpmn:sequenceFlow id="gateway1-task3" name="Route is less than one kilometer" sourceRef="gateway1" targetRef="task3" />
    <bpmn:sequenceFlow id="task4-end2" sourceRef="task4" targetRef="end2" />
    <bpmn:sequenceFlow id="gateway1-task4" name="Route is one kilometer or more" sourceRef="gateway1" targetRef="task4" />
  </bpmn:process>
</bpmn:definitions>
```



```

<bpmndi:BPMNDiagram id="BPMNDiagram_Process_1">
  <bpmndi:BPMNPlane id="BPMNPlane_Process_1" bpmnElement="Process_1">
    <bpmndi:BPMNShape id="start_di" bpmnElement="start">
      <dc:Bounds x="57" y="52" width="36" height="36" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="task1_di" bpmnElement="task1">
      <dc:Bounds x="175" y="30" width="100" height="80" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="task2_di" bpmnElement="task2">
      <dc:Bounds x="325" y="30" width="100" height="80" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="gateway1_di" bpmnElement="gateway1" isMarkerVisible="true">
      <dc:Bounds x="500" y="45" width="50" height="50" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="task3_di" bpmnElement="task3">
      <dc:Bounds x="625" y="30" width="100" height="80" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="end1_di" bpmnElement="end1">
      <dc:Bounds x="807" y="52" width="36" height="36" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="task4_di" bpmnElement="task4">
      <dc:Bounds x="625" y="170" width="100" height="80" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNShape id="end2_di" bpmnElement="end2">
      <dc:Bounds x="807" y="192" width="36" height="36" />
    </bpmndi:BPMNShape>
    <bpmndi:BPMNEdge id="start-task1_di" bpmnElement="start-task1">
      <di:waypoint x="93" y="70" />
      <di:waypoint x="175" y="70" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="task1-task2_di" bpmnElement="task1-task2">
      <di:waypoint x="275" y="70" />
      <di:waypoint x="325" y="70" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="task2-gateway1_di" bpmnElement="task2-gateway1">
      <di:waypoint x="425" y="70" />
      <di:waypoint x="500" y="70" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="gateway1-task3_di" bpmnElement="gateway1-task3">
      <di:waypoint x="550" y="70" />
      <di:waypoint x="625" y="70" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="gateway1-task4_di" bpmnElement="gateway1-task4">
      <di:waypoint x="525" y="95" />
      <di:waypoint x="525" y="210" />
      <di:waypoint x="625" y="210" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="task3-end1_di" bpmnElement="task3-end1">
      <di:waypoint x="725" y="70" />
      <di:waypoint x="807" y="70" />
    </bpmndi:BPMNEdge>
    <bpmndi:BPMNEdge id="task4-end2_di" bpmnElement="task4-end2">
      <di:waypoint x="725" y="210" />
      <di:waypoint x="807" y="210" />
    </bpmndi:BPMNEdge>
  </bpmndi:BPMNPlane>
</bpmndi:BPMNDiagram>
</bpmn:definitions>

```

